

Universidad Mariano Gálvez De Guatemala

Ingeniería en sistemas

Curso: Análisis de sistemas

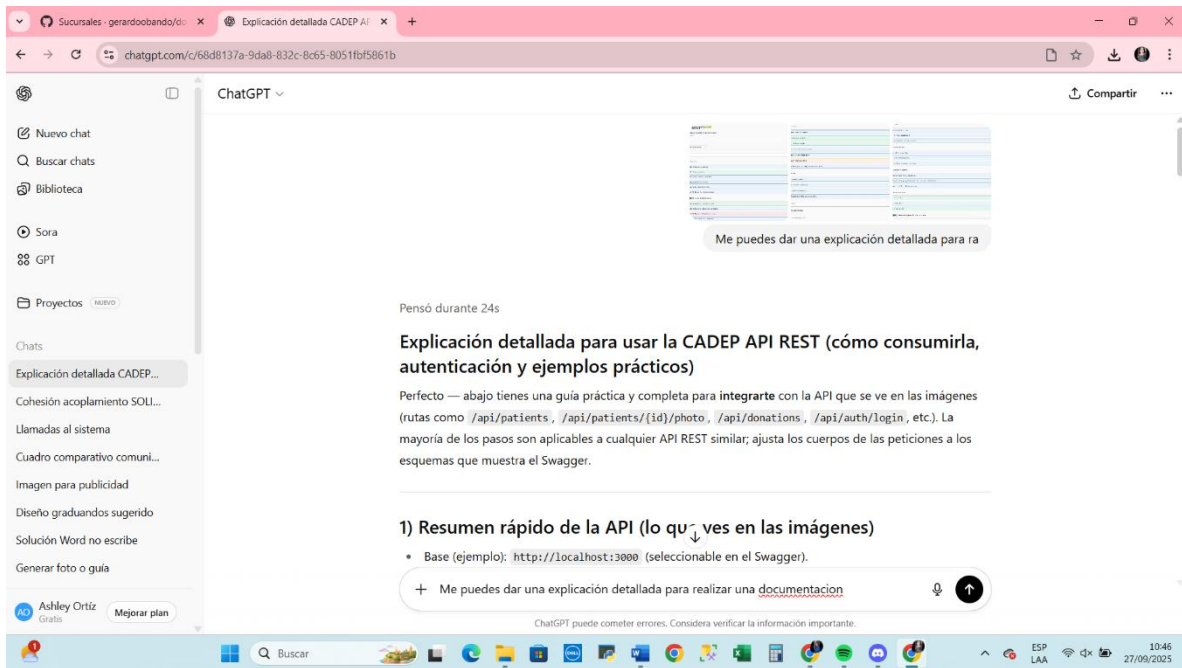
Catedrática: Ing. Richard Sazvin



Documentación CADEP

Sergio Arriaza 1890-22-1387

Ashley Ortiz 1890-18-17558



CADEP API REST 1.0.0 OAS 3.0	
API REST para el sistema de gestión de pacientes y donaciones de CADEP	
Contact Equipo de Desarrollo CADEP	
MIT	
Servers	
http://localhost:3000 - Servidor de desarrollo	
Filter by tag	
Pacientes Gestión de pacientes	
GET	/api/patients Obtener todos los pacientes
POST	/api/patients Crear nuevo paciente
GET	/api/patients/stats Estadísticas de pacientes
GET	/api/patients/search Buscar pacientes
GET	/api/patients/{id} Obtener paciente por ID
GET	/api/patients/{id}/similar Pacientes similares
Fotos de Pacientes Gestión de fotos de pacientes	
POST	/api/patients/{id}/photo Subir foto del paciente
GET	/api/patients/{id}/photo Obtener información de la foto
DELETE	/api/patients/{id}/photo Eliminar foto del paciente
GET	/api/patients/photos/test Test de rutas de fotos

Donaciones	Gestión de donaciones	^
GET	/api/donations	Obtener todas las donaciones
POST	/api/donations	Crear donación completa
POST	/api/donations/quick	Donación rápida
GET	/api/donations/stats	Estadísticas de donaciones
GET	/api/donations/{id}	Obtener donación por ID
PUT	/api/donations/{id}	Actualizar donación
GET	/api/donations/patient/{patientId}	Donaciones por paciente
Donadores	Gestión de donadores	^
GET	/api/donors	Obtener todos los donadores
GET	/api/donors/stats	Estadísticas de donadores
GET	/api/donors/{id}	Obtener donador por ID
GET	/api/donors/{id}/donations	Donaciones de un donador
Sucursales	Gestión de sucursales	^
Áreas Médicas	Gestión de áreas médicas	^
Ubicaciones	Gestión de ubicaciones geográficas	^

Sucursales	Gestión de sucursales	^
GET	/api/branches	Obtener todas las sucursales
GET	/api/branches/{id}	Obtener sucursal por ID
GET	/api/branches/{id}/patients	Pacientes por sucursal
Áreas Médicas	Gestión de áreas médicas	^
GET	/api/areas	Obtener todas las áreas médicas
GET	/api/areas/{id}	Obtener área médica por ID
GET	/api/areas/{id}/patients	Pacientes por área médica
Ubicaciones	Gestión de ubicaciones geográficas	^
GET	/api/locations/departments	Obtener departamentos
GET	/api/locations/departments/{id}/municipalities	Municipios por departamento
GET	/api/locations/municipalities	Obtener municipios
Autenticación	Autenticación y autorización	^
POST	/api/auth/login	Iniciar sesión
GET	/api/auth/verify	Verificar token
POST	/api/auth/refresh	Renovar token
Imágenes de Pacientes	Gestión de imágenes adicionales de pacientes	^

Explicación detallada para usar la CADEP API REST (cómo consumirla, autenticación y ejemplos prácticos)

Perfecto — abajo tienes una guía práctica y completa para **integrarte** con la API que se ve en las imágenes (rutas como `/api/patients`, `/api/patients/{id}/photo`, `/api/donations`, `/api/auth/login`, etc.). La mayoría de los pasos son aplicables a cualquier API REST similar; ajusta los cuerpos de las peticiones a los esquemas que muestra el Swagger.

Resumen rápido de la API (lo que ves en las imágenes)

- Base (ejemplo): `http://localhost:3000` (seleccionable en el Swagger).
 - Recursos principales visibles:
 - **Pacientes:** `/api/patients`, `/api/patients/{id}`, `/api/patients/search`, `/api/patients/{id}/similar`
 - **Fotos de pacientes:** `/api/patients/{id}/photo` (POST, GET, DELETE)
 - **Donaciones:** `/api/donations`, `/api/donations/quick`, `/api/donations/{id}`, `/api/donations/patient/{patientId}`
 - **Donadores:** `/api/donors`, `/api/donors/{id}/donations`
 - **Sucursales / Áreas / Ubicaciones:** `/api/branches`, `/api/areas`, `/api/locations/...`
 - **Autenticación:** `/api/auth/login`, `/api/auth/verify`, `/api/auth/refresh`
 - Swagger UI te muestra documentación interactiva con métodos, parámetros y (en muchos casos) esquemas de request/response.
-

2) Autenticación (cómo funciona y cómo usarla)

1. **Login:** envías credenciales a POST `/api/auth/login`. Respuesta típica: un access token (ej. JWT) y quizá refresh token.
 2. **Enviar token:** luego incluyes el encabezado:
 3. Authorization: Bearer <ACCESS_TOKEN>
 4. **Renovar:** si hay refresh, usar POST `/api/auth/refresh`.
 5. **Swagger:** en la UI suele haber un botón **Authorize** o íconos de candado junto a rutas protegidas; pega Bearer <token> ahí para probar desde Swagger.
-

3) Tipos de contenido y formatos

- `application/json` → para GET/POST/PUT con cuerpo JSON (crear paciente, actualizar donación).

- multipart/form-data → para subir archivos (ej. POST /api/patients/{id}/photo). Campo habitual: photo o file (revisar parámetros en Swagger).
-

4) Buenas prácticas: parámetros, paginación y errores

- **Path params:** {id}, {patientId} → parte de la URL.
 - **Query params:** búsquedas/paginación e.g. ?page=1&limit=25&q=juan (revisa si la API soporta estos parámetros).
 - **Manejo de errores:** comprobar códigos HTTP:
 - 200/201 → OK / creado
 - 400 → petición inválida (falta campo)
 - 401 → no autorizado (token faltante o expirado)
 - 404 → recurso no encontrado
 - 500 → error del servidor
 - **Logs y reintentos:** implementar reintentos exponenciales para errores transitorios y timeouts razonables.
-

5) Ejemplos prácticos

a) Obtener token (curl)

```
curl -X POST "http://localhost:3000/api/auth/login" \  
-H "Content-Type: application/json" \  
-d '{"username":"miusuario","password":"miclave"}
```

Respuesta esperada (ejemplo genérico):

```
{  
  "accessToken": "eyJhbGciOiJI...",  
  "refreshToken": "..."  
}
```

b) Llamada GET con token (curl)

TOKEN="eyJhbGciOiJI..."

```
curl -X GET "http://localhost:3000/api/patients" \  
-H "Authorization: Bearer $TOKEN"
```

c) Crear paciente (POST) — ejemplo JSON (adapta según Swagger)

```
curl -X POST "http://localhost:3000/api/patients" \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "firstName": "María",  
  "lastName": "Pérez",  
  "birthDate": "1990-05-10",  
  "idNumber": "12345678",  
  "phone": "50212345678",  
  "address": "Ciudad",  
  "medicalAreaId": 3  
}'
```

d) Subir foto de paciente (multipart/form-data)

```
curl -X POST "http://localhost:3000/api/patients/123/photo" \  
-H "Authorization: Bearer $TOKEN" \  
-F "photo=@/ruta/a/foto.jpg"
```

e) Obtener foto / eliminar foto

Obtener info de la foto

```
curl -X GET "http://localhost:3000/api/patients/123/photo" -H "Authorization: Bearer $TOKEN"
```

Eliminar

```
curl -X DELETE "http://localhost:3000/api/patients/123/photo" -H "Authorization: Bearer $TOKEN"
```

f) Ejemplo en Python (requests)

```
import requests
```

```
BASE = "http://localhost:3000"
```

1) Login

```
r = requests.post(f'{BASE}/api/auth/login', json={"username": "miuser", "password": "pass"})
```

```
token = r.json().get("accessToken")
```

```
headers = {"Authorization": f"Bearer {token}"}
```

2) Listar pacientes

```
r = requests.get(f"{BASE}/api/patients", headers=headers)
print(r.status_code, r.json())
```

3) Subir foto

```
files = {"photo": open("foto.jpg", "rb")}
r = requests.post(f"{BASE}/api/patients/123/photo", headers=headers, files=files)
print(r.status_code, r.json())
```

g) Ejemplo en Node (fetch)

```
const fetch = require('node-fetch');
const BASE = "http://localhost:3000";
```

```
async function getPatients(token) {
  const res = await fetch(`${BASE}/api/patients`, {
    headers: { Authorization: `Bearer ${token}` }
  });
  return res.json();
}
```

6) Cómo probar rápidamente desde Swagger UI

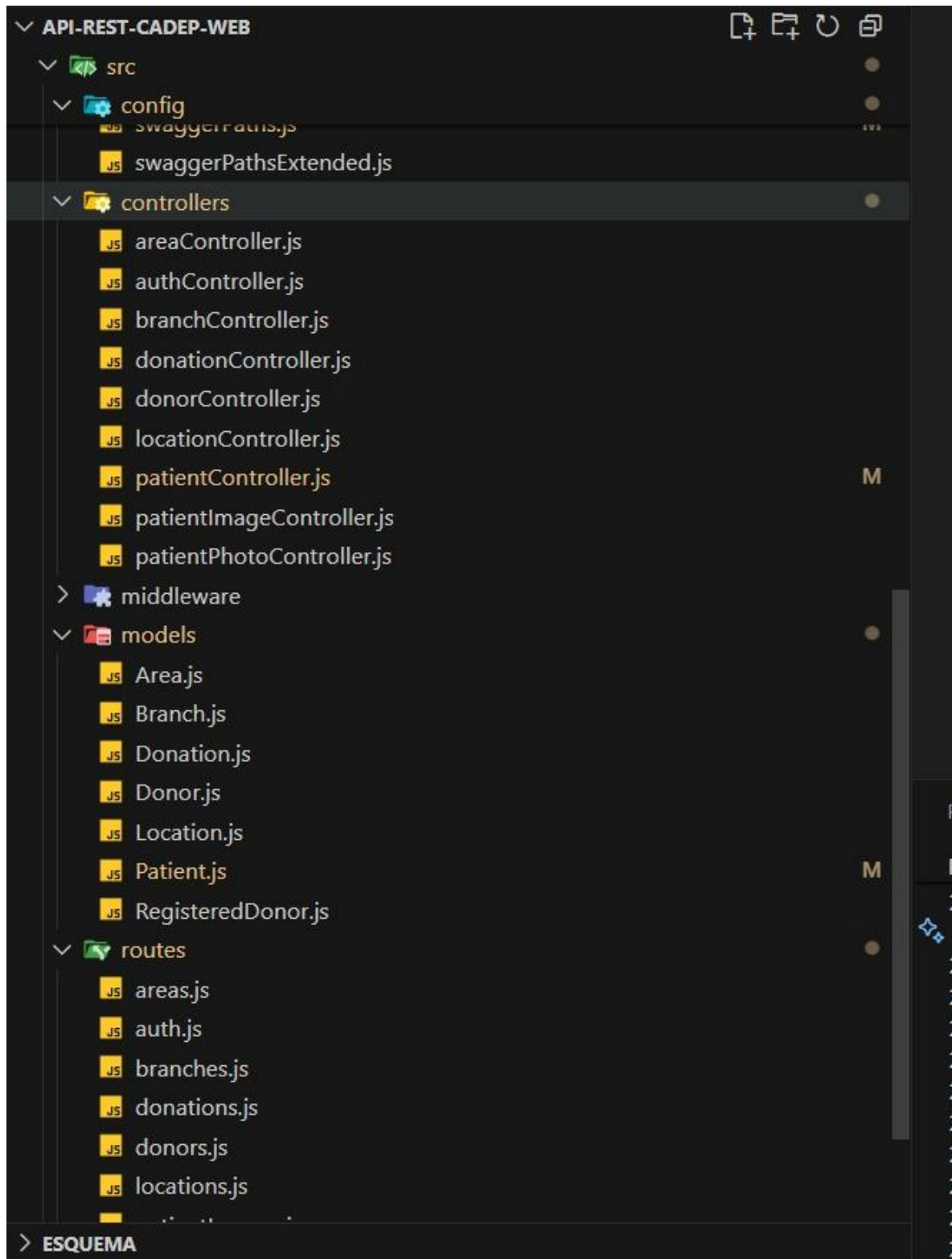
1. Selecciona el **server** correcto (ej. `http://localhost:3000`).
 2. Busca POST `/api/auth/login`, haz **Try it out**, envía credenciales.
 3. Copia el token de la respuesta.
 4. Usa el botón **Authorize** (o el candado), pega `Bearer <token>` (o solo el token si la UI pide eso).
 5. Ahora puedes probar endpoints protegidos (GET/POST/PUT).
 6. Para subir archivos, en la sección del endpoint activa **Try it out** y adjunta el archivo (si Swagger lo soporta).
-

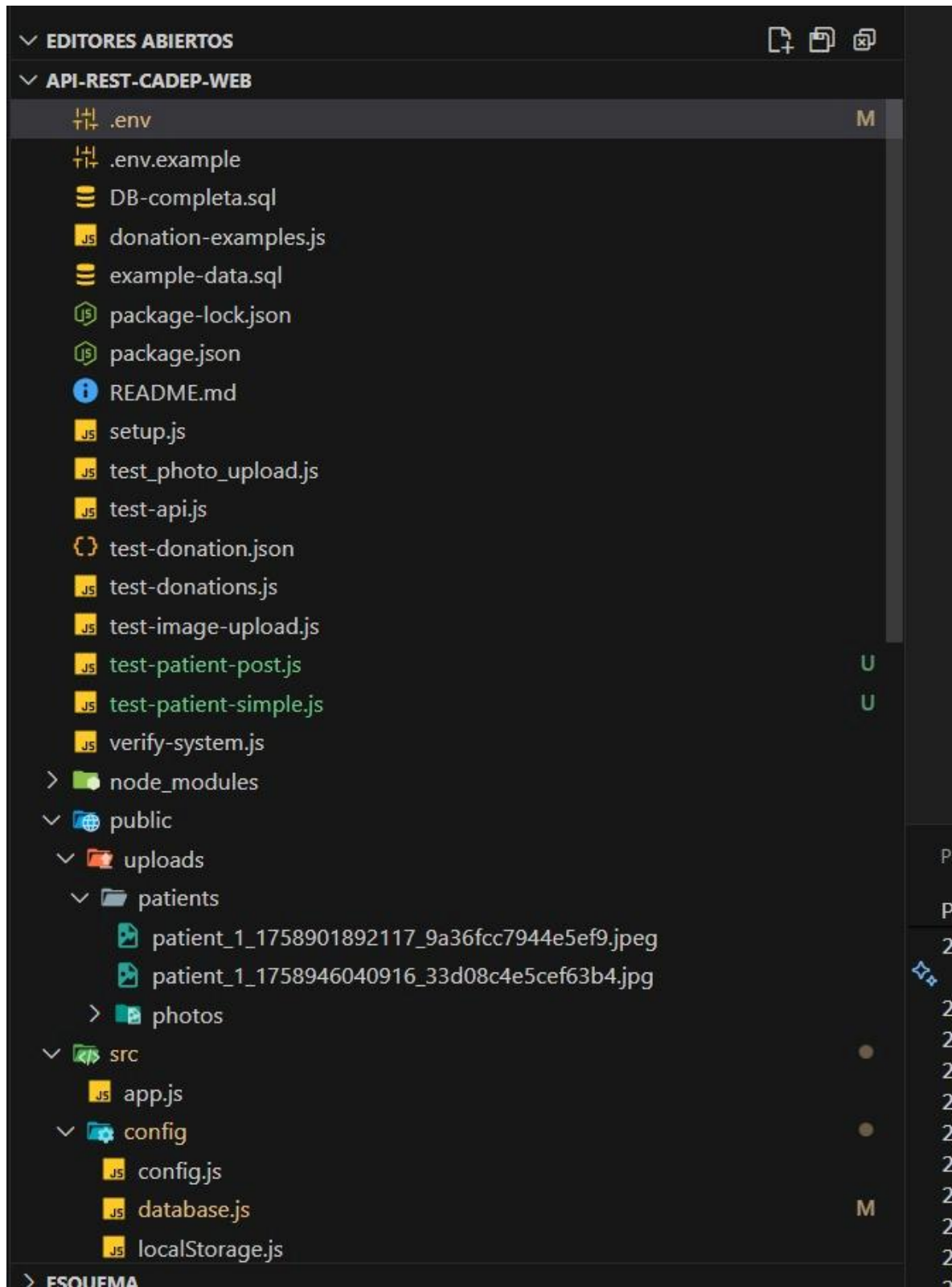
7) Errores comunes y soluciones rápidas

- **401 después de login:** token no enviado o mal formato (Authorization: Bearer ...).
 - **Campos faltantes (400):** revisar el body con los campos required en Swagger.
 - **CORS en front:** si llamas desde browser, puede bloquearse por CORS; la solución está en backend (habilitar orígenes).
 - **Multipart fail:** asegurarse Content-Type lo maneje curl -F o requests files=... (no poner Content-Type: application/json).
 - **Ruta incorrecta:** revisar que haya /api/ prefijo (en las imágenes todas rutas empiezan por /api/...).
-

8) Checklist para integrar la API en tu app

- Base URL en variable de entorno
- Manejo seguro de tokens (almacenar en memoria/secure storage; no en localStorage si es sensible)
- Reintentos y timeouts
- Manejo de errores y mostrar mensajes útiles al usuario
- Validación de campos antes de enviar
- Probar con Swagger UI y con Postman
- Implementar refresh token (si existe)
- Logs/monitorización de fallos





Vista general del repositorio (lo que se ve en las capturas)

- `.env` / `.env.example` — variables sensibles (BD, puerto, JWT, etc.). Antes de correr el proyecto copia `.env.example` → `.env` y ajusta valores.
- `DB-completa.sql` / `example-data.sql` — dump de la base de datos; útil para crear la BD y datos de prueba.
- `package.json` — dependencias y scripts (`start`, `dev`, `test`, etc.).
- `public/uploads/...` — carpeta donde se guardan archivos subidos (en la captura: `patients` y `photos`).
- `src/` — código fuente:
 - `app.js` — punto de entrada (configura Express, middleware, rutas, estáticos, swagger).
 - `config/` — configuración: DB, swagger, `localStorage`, etc.
 - `controllers/` — la lógica que maneja las rutas (`patientController`, `authController`, etc.).
 - `models/` — modelos (`Area`, `Branch`, `Donation`, `Patient`, etc). Probablemente usa un ORM (Sequelize u otro).
 - `routes/` — definiciones de rutas (`areas.js`, `branches.js`, `patients.js`, etc).
 - `middleware/` — `auth`, validaciones, manejo de errores, etc.
- `test-*.js` — pruebas o scripts para probar endpoints (ya hay varios archivos de prueba en la raíz).

2) Flujo general de la API (cómo se conecta todo)

1. `src/app.js` inicializa Express, aplica middlewares (`bodyParser`, `CORS`, `cors`, autenticación), registra rutas con `app.use('/api/patients', require('./routes/patients'))`, y expone `public/uploads` como estático `app.use('/uploads', express.static(path.join(__dirname, '..', 'public', 'uploads')))`.
2. Cada archivo en `routes/` mapea endpoints a funciones del controlador:
3. `// routes/patients.js` (ejemplo)
4. `const express = require('express');`
5. `const router = express.Router();`
6. `const patientController = require('../controllers/patientController');`
- 7.
8. `router.get('/', patientController.index);` // listar / buscar
9. `router.post('/', patientController.store);` // crear
10. `router.get('/:id', patientController.show);` // ver

11. `router.put('/:id', patientController.update); // actualizar`
12. `router.delete('/:id', patientController.destroy); // eliminar`
- 13.
14. `module.exports = router;`
15. `controllers/*` contiene la lógica concreta (queries, validaciones, manejo de archivos). Los `models/*` representan tablas/entidades y definen relaciones.

3) Configuración de la base de datos

- `config/database.js` normalmente exporta la conexión (host, user, password, database) usando las variables en `.env`.
- Importa `DB-completa.sql` en tu servidor de base de datos para tener tablas y datos de ejemplo.
- Si los modelos usan Sequelize, en cada `models/*.js` verás `sequelize.define(...)` o `class Patient extends Model` — así confirmas el ORM.

4) Manejo de imágenes/subidas

- Carpeta: `public/uploads/patients/` — ahí se guardan los JPG/PNG con nombres únicos (ej. `patient_1_175890...jpg`).
- Normalmente se usa `multer` para recibir `multipart/form-data` y guardarlo en esta carpeta.
- `patientImageController` o `patientPhotoController` se encargan de:
 - subir la imagen y asociar la ruta al registro `Patient`.
 - servir la ruta pública (la URL será algo como `https://miapi/uploads/patients/archivo.jpg`).
 - en `delete`: eliminar el archivo del sistema cuando se borre el paciente o se reemplace la foto.

Ejemplo de `multer` (snippet):

```
const multer = require('multer');
```

```
const path = require('path');
```

```
const storage = multer.diskStorage({
```

```
  destination: (req, file, cb) => cb(null, path.join(__dirname, '..', '..', 'public', 'uploads', 'patients')),
```

```
  filename: (req, file, cb) => {
```

```
    const ext = path.extname(file.originalname);
```

```

    cb(null, `patient_${req.params.id || 'new'}_${Date.now()}${ext}`);
  }
});
const upload = multer({ storage });
module.exports = upload;

```

5) Ejemplos prácticos: endpoints y consultas comunes

A) Listar / buscar pacientes (con filtros: name, branch, rango de fechas, paginación)

Asumo Sequelize; si usas otro ORM la idea es análoga.

// controllers/patientController.js (ejemplo)

```

const { Patient, Branch } = require('../models');
const { Op } = require('sequelize');

```

```

exports.index = async (req, res) => {
  try {
    const { name, branchId, fromDate, toDate, page = 1, limit = 20 } = req.query;
    const where = {};

    if (name) where.name = { [Op.iLike]: `%${name}%` }; // iLike en Postgres
    if (branchId) where.branchId = branchId;
    if (fromDate || toDate) {
      where.createdAt = {};
      if (fromDate) where.createdAt[Op.gte] = new Date(fromDate);
      if (toDate) where.createdAt[Op.lte] = new Date(toDate);
    }

```

```

    const patients = await Patient.findAndCountAll({
      where,
      include: [{ model: Branch }],
      limit: Number(limit),
      offset: (page - 1) * limit,

```

```

    order: [['createdAt', 'DESC']],
  });

  res.json({ total: patients.count, data: patients.rows });
} catch (err) {
  console.error(err);
  res.status(500).json({ message: 'Error al obtener pacientes' });
}
};

```

Notas:

- Usa Op.like/Op.iLike depende de la DB.
- findAndCountAll facilita paginación.
- Sanitiza los inputs para evitar inyección.

B) Crear paciente (y subir foto separada)

- POST /patients crea el registro (sin foto).
- POST /patients/:id/photo con multipart/form-data sube foto y actualiza patient.photoPath.

C) Actualizar y eliminar paciente (borrar foto física)

Cuando elimines paciente, borra el archivo en disco:

```
const fs = require('fs');
```

```
const path = require('path');
```

```
exports.destroy = async (req, res) => {
```

```
  try {
```

```
    const patient = await Patient.findByPk(req.params.id);
```

```
    if (!patient) return res.status(404).json({ message: 'No existe' });
```

```
    // eliminar archivo si existe
```

```
    if (patient.photoPath) {
```

```
      const fullPath = path.join(__dirname, '..', '..', 'public', patient.photoPath);
```

```

    fs.unlink(fullPath, (err) => { if (err) console.warn('No se pudo borrar archivo', err); });
  }

  await patient.destroy();
  res.json({ message: 'Paciente eliminado' });
} catch (err) {
  res.status(500).json({ message: 'Error al eliminar' });
}
};

```

6) Rutas / controllers que ya tienes (según capturas)

- areaController.js ↔ routes/areas.js (CRUD de áreas)
- authController.js ↔ routes/auth.js (login, registro, JWT)
- branchController.js ↔ routes/branches.js
- donationController.js ↔ routes/donations.js
- donorController.js ↔ routes/donors.js
- locationController.js ↔ routes/locations.js
- patientController.js ↔ routes/patients.js
- patientImageController.js / patientPhotoController.js ↔ endpoints para imágenes

7) Archivos de utilidad y documentación

- config/swagger*.js — si quieres ver la documentación de la API swaggger (UI). Ruta típica: /api-docs.
- README.md — instrucciones de uso (revisa si hay pasos para correr).
- test-*.js — scripts que usan fetch/axios para probar endpoints (útiles para ver payloads esperados).

8) Pasos rápidos para levantar el proyecto localmente

1. git clone ...
2. cp .env.example .env y configurar valores.
3. Crear BD e importar DB-completa.sql.
4. npm install
5. npm run dev o node src/app.js

6. Abrir `http://localhost:PORT/api/...` y `http://localhost:PORT/uploads/...` para las imágenes.

9) Errores comunes y cómo detectarlos

- **Credenciales BD incorrectas** → revisa `.env` y `config/database.js`.
- **Carpeta public/uploads no existe** → crea y da permisos.
- **Multer no configurado** → revisar tipo de input en el frontend (`name="photo"`).
- **Rutas montadas mal** → revisar `app.js` para `app.use('/api/patients', patientsRoutes)`.
- **Problemas CORS** → agregar `app.use(cors())`.
- **Fallo en relaciones** → verificar `models/index.js` donde se definen `associations`.

10) Recomendaciones de mejora

- Separar lógica en `services/` para no sobrecargar `controllers`.
- Validar entradas con `Joi` o `express-validator`.
- Usar transacciones al crear recursos relacionados.
- Manejar errores centralizados en `middleware`.
- Usar migraciones y `seeders` (si usas `Sequelize`).
- Añadir pruebas automatizadas (`jest/supertest`).
- Sanitizar y limitar tamaño de archivos subidos.
- No commitear `.env` ni archivos sensibles.